Dipartimento di Elettronica e Informazione

Politecnico di Milano
prof. ***Carlo Ghezzi***
prof. ***Angelo Morzenti***
prof. ***Pierluigi San Pietro***

20133 Milano (Italia)
Piazza Leonardo da Vinci, 32
Tel. (39) 02-2399.3400
Fax (39) 02-2399.3411

II Prova di Ingegneria del Software

28 Giugno 2004

Cognome

Nome

Matricola

Sezione (segnarne una) Ghezzi □, Morzenti □, SanPietro □

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h30m.
6. Punteggio totale a disposizione: 13/30.

Valutazione:
Esercizio 1: ……..

Esercizio 2: ……..

Esercizio 3: ……..

Esercizio 1 (punti 4)
Si consideri la seguente specifica informale sotto riportata che definisce le regole riguardanti il superamento dell'esame di Ingegneria del Software.

> "Verranno svolte due prove intermedie. Ogni prova intermedia assegna un massimo di 13 punti ed è considerata valida se lo studente ottiene almeno un punteggio minimo di 6 punti per la I prova e di 6 punti per la II prova; chi ottiene un punteggio inferiore a quello minimo in una prova è obbligato a ripeterla nelle prove di recupero. La ripetizione riguarda comunque entrambe le prove; pertanto il mancato raggiungimento della soglia minima in una delle due prove richiede la partecipazione a una prova di recupero che riguarda l'intero programma del corso.
> L'attività svolta in laboratorio permette di ottenere un massimo di 4 punti, ed è considerata sufficiente se lo studente ottiene almeno 2 punti. Non è previsto il recupero del laboratorio. Pertanto in caso di valutazione insufficiente lo studente dovrà ripetere il corso nell'anno accademico successivo: saranno annullati gli eventuali risultati ottenuti durante le prove in itinere e non sarà possibile partecipare agli appelli.
> Per superare l'esame è inoltre necessario che la somma dei punteggi delle due prove in itinere sia almeno di 16 punti sui 26 disponibili e che il risultato complessivo (che comprende anche il voto relativo all'attività di laboratorio) sia almeno 18; lo studente che non soddisfa le precedenti condizioni dovrà recuperare entrambe le prove in un appello a propria scelta."

1. Utilizzando opportune formule che definiscono le pre e post-condizioni, si fornisca la specifica di un'astrazione procedurale che in input riceve tre valori: P1, P2, L, che rappresentano i voti ottenuti nella I e nella II prova intermedia e nel laboratorio, rispettivamente. In output viene fornito un valore intero che rappresenta il voto conseguito (se sufficiente), oppure 0, se l'allievo deve sostenere il recupero, o -1 se deve ripetere;

public static int risultato(int P1, P2, L)
//REQUIRES: 0 ≤P1 ≤ 13 && 0 ≤P2 ≤ 13 && 0 ≤L ≤ 4
//EFFECTS: if (L < 2) return -1
//else if (P1 < 6 || P2 < 6 || P1 + P2 < 16) -> return 0
//else return P1+P2+L

2. In base alla specifica fornita, si definiscano dati a supporto del test funzionale (black-box testing) in base al criterio di copertura delle combinazioni proposizionali e i risultati attesi per ciascun dato.

< P1, P2, L >
< 7, 7, 0 > (per L<2), valore atteso: -1
< 3, 4, 3 > (sia per P1<6 che per P2<6), valore atteso: 0
< 3, 12, 3 > (per P1<6 ma P2>=6, P1+P2>=16), valore atteso: 0
< 7, 7, 3 > (per P1 + P2 < 16) valore atteso: 0
< 10, 9, 3 > valore atteso: 22

3. Si identifichino almeno 5 dati di test corrispondenti a *valori limite* dei parametri dell'astrazione.

< P1, P2, L >
< 7, 7, 1 > (valore limite di L<2), valore atteso: -1
< 5, 5, 2 > valore atteso: 0
< 6, 6, 2 > valore atteso: 0
< 7, 8, 2 > valore atteso: 0
< 8, 8, 2 > valore atteso: 18
< 13, 13, 4 > valore atteso: 30

Esercizio 2 (punti 6)

La classe astratta Coda specifica le operazioni per la manipolazioni di una coda FIFO di Object: insert (inserisce in coda un Object), remove (estrae un Object dalla coda, eliminandolo, e lo restituisce come risultato), length (lunghezza della coda). L'iteratore Elements restituisce un generatore che consente di iterare su tutti gli oggetti in coda. Le operazioni ritornano un'eccezione (NullPointerException) nel caso in cui l'oggetto a cui si applicano sia null. L'operazione di inserimento può, in alcuni casi che non vengono dettagliati, sollevare un'eccezione di tipo OggettoNonValido anziché inserire l'oggetto. Non vengono generate altre eccezioni dalle operazioni (tranne l'eccezione CodaVuota nel caso si cerchi di estrarre un oggetto da una coda vuota); i metodi che implementano le operazioni definiscono inoltre funzioni totali.

1. Considerando la seguente implementazione parziale di Coda, si completino le operazioni con la loro specifica (OVERVIEW, REQUIRES, EFFECTS, MODIFIES)

```
public abstract class Coda {
……
      protected int tot;
      public Coda() {tot = 0};
……
……
      public abstract void insert (Object x) throws OggettoNonValido;
……
……
      public abstract Object remove() throws CodaVuota;
……
……
      public abstract Iterator Elements();
……
……
      public int length() {return tot;}
……
……
}
```

```
public abstract class Coda {
//OVERVIEW: Coda è un contenitore di Object. Tipo Mutabile. L'inserimento di un oggetto nel
contenitore potrebbe non essere sempre possibile. L'estrazione rimuove gli oggetti nello stesso
ordine del loro avvenuto inserimento (seguendo cioe' una politica FIFO).
      //Un oggetto tipico è [o1, o2, …, on], dove o1 e' l'elemento inserito da piu' tempo.
      protected int tot;
      public Coda() {tot = 0};
      //REQUIRES: true
      //EFFECTS: inizializza this alla coda vuota

      public abstract void insert (Object x) throws OggettoNonValido;
      //REQUIRES: true
      //EFFECTS: // posto this = [o1, o2, …, on],
      //if x != null /then  (this_post = [o1, o2, …, on, x] || solleva OggettoInvalido),
      //else solleva NullPointerException

      public abstract Object remove ( ) throws CodaVuota;
      //REQUIRES: true
      //EFFECTS: // posto this = [o1, o2, …, on],
      //se n>0, this_post = [o2, …, on] e ret_val = o1, altrimenti solleva CodaVuota

      public abstract Iterator Elements();
      //REQUIRES: true
      //EFFECTS: restituisce un generatore che restiuisce gli elementi di this nell'ordine o1, .., on.

      public int length() {return tot;}
      //REQUIRES: true
      //EFFECTS: restituisce la lunghezza della coda. Formalmente, se this = [o1, o2, …, on], allora
      //ret_val = n, e this_post = this_pre
```

}

(**continua esercizio 2)**

2. Si vogliono definire due sottoclassi concrete di Coda, chiamate rispettivamente CodaIllimitata e CodaFinita, che rappresentano, rispettivamente, le code che possono crescere arbitrariamente di lunghezza e le code di lunghezza finita e predeterminata. Nel secondo caso, si assume che la lunghezza massima delle code sia definita da una costante statica privata e che il tentativo di inserimento di un Object in una coda piena generi un'eccezione checked CodaPiena. CodaIllimitata e CodaFinita rispettano il principio di sostituibilità? Motivare la risposta facendo riferimento a una precisa descrizione della specifica di CodaFinita e di CodaIllimitata.

```
public class CodaFinita extends Coda {
    ……
    ……
    ……
    ……
    ……
    ……
    ……
    ……
    ……
}
```

```
public class CodaIllimitata extends Coda {
    ……
    ……
    ……
    ……
    ……
    ……
    ……
    ……
    ……
}
```

public class CodaFinita extends Coda {
//OVERVIEW: CodaFinita è una Coda di dimensione massima MAX uguale per tutte le code
//Al raggiungimento del numero massimo di elementi, l'inserimento restituisce eccez. CodaPiena
public abstract void insert (Object x) throws OggettoNonValido. CodaPiena
//EFFECTS: if la dimensione massima di this è maggiore di MAX, throw CodaPiena else....
}
public class CodaIllimitata extends Coda {
//OVERVIEW: CodaIllimitata è una Coda la cui dimensione può essere qualunque
}

La classe CodaIllimitata rispetta il principio di sostituzione, la classe CodaFinita no. CodaIllimitata non modifica la specifica dei metodi classe Coda, quindi le regole delle segnature e dei metodi sono banalmente verificate. L'overview di CodaIllimitata impone solo la proprietà aggiuntiva che le dimensioni della coda siano qualunque, ma non nega nessuna delle proprietà specificate nell'Overview della sopraclasse (come la politica FIFO, la mutabilità, ecc.). La classe CodaFinita invece aggiunge un'eccezione (checked) alla signature del metodo insert, in violazione della regola delle signature.

3. Si consideri la classe AltraCodaFinita, che ha la stessa definizione della classe CodaFinita ma con la seguente modifica: quando la coda è piena, il metodo insert genera un'eccezione OggettoNonValido (anziché l'eccezione CodaPiena). La classe AltraCodaFinita soddisfa il principio di sostituibilità? Anche in questo caso giustificare la risposta, aiutandosi con una precisa descrizione della specifica di CodaFinita.

```
public class AltraCodaFinita extends Coda {
    ……
    ……
    ……
    ……
    ……
    ……
    ……
    ……
    ……
}
```

Se il metodo solleva un'eccezione di tipo OggettoNonValido, allora la regola delle signature viene rispettata. Anche la regola dei metodi viene rispettata, dal momento che il metodo resta totale (precondizioni invariate) e la postcondizione di Coda.insert consente di sollevare ad arbitrio l'eccezione: AltraCodaFinita "raffina" questa postcondizione specificando più in dettaglio un caso in cui viene sollevata l'eccezione, OggettoNonValido ed è quindi più forte (ossia, implica) la postcondizione di Coda.insert. Per quanto riguarda, infine, la regola delle proprietà, la classe AltraCodaFinita ha più proprietà (la limitatezza della dimensione della coda) della classe Coda, ma rispetta ancora le proprietà dell'overview di Coda (politica FIFO e mutabilità). Pertanto, anche la regola delle proprietà è soddisfatta, e la classe AltraCodaFinita rispetta il principio di sostituibilità.

4. Si consideri la classe TerzaCodaFinita, la cui specifica è identica a quella di CodaFinita, con la seguente modifica: quando la coda è piena, il metodo insert non fa nulla. Dire, anche in questo caso, se TerzaCodaFinita rispetta o non rispetta il principio di sostituibilità, giustificando come al solito la risposta.

```
public class TerzaCodaFinita extends Coda {
    ……
    ……
    ……
    ……
    ……
    ……
}
```

Stavolta il metodo TerzaCodaFinita.insert ha una postcondizione che non rispetta quella specificata nella classe Coda: quest'ultima dice che insert può solo inserire un oggetto nella coda oppure non inserirlo e sollevare un eccezione. Dal momento che TerzaCodaFinita.insert in certi casi non inserisce un oggetto in coda, ma in tali casi non solleva l'eccezione corrispondente, la condizione ensure no less sulle postcondizioni del metodo insert viene violata.

(**continua esercizio 2)**

5. Si tratteggi l'implementazione della classe CodaIllimitata volendo che essa sia utilizzabile per definire un'astrazione polimorfa, usabile per definire code non limitate in dimensioni di elementi di un qualunque tipo. Si vuole verificare che gli elementi inseriti in una coda illimitata siano tutti dello stesso tipo. A tale scopo si adotti l'approccio *element subtype*, definendo un'interfaccia chiamata Accodabile con un metodo confrontaClasse; il metodo insert, se la coda in cui avviene l'inserzione non è vuota, invoca il metodo confrontaClasse su uno degli elementi già presenti in coda, passando come parametro l'oggetto di tipo Accodabile che si sta tentando di inserire. Nel caso che l'oggetto abbia tipo scorretto, questo non viene inserito nella coda, e viene sollevata un'eccezione di tipo OggettoNonValido. Definire una classe StringaAccodabile, che contiene un dato di tipo String e implementa Accodabile, codificando in dettaglio il suo metodo confrontaClasse.

```
public class CodaIllimitata extends Coda {
    ……
    public void insert ……..
    ……
    ……
    ……
    ……
}

public interface Accodabile {
    public boolean confrontaClasse(Object x);
}

public class StringaAccodabile implements Accodabile {
    private String s;
    ……
    ……
    ……
    ……
    ……
    ……
}
public class Codalllimitata extends Coda {
    //OVERVIEW: Codalllimitata di elementi omogenei
    //Rep:
    private Vector coda;
    //AF: AF(c) =  [c.coda.get(0) ... c.coda.get(coda.size()-1)]
    //RI(c) = c.coda != null
    public void insert(Object x) throws OggettoNonValido {
        if ( !(x instanceof Accodabile) ||
            (!c.isEmpty()   &&   !((Accodabile) x).confrontaClasse(coda.firstElement()))
            throw new OggettoNonValido();

        coda.add(x); //…a questo punto può inserire x nel vettore coda
    }
}
public class StringaAccodabile implements Accodabile {
    private String s;
    public StringaAccodabile(String s) {this.s = s;}
    public String contenuto() {return s;}
    public boolean confrontaClasse (Object x) {
        if !(x instanceof  StringaAccodabile)) return false;
        return true;
    }
    NB: L'implementazione di Codalllimitata qui riportata è più completa di quanto strettamente richiesto
    dall'esercizio.
```

```
}
```

Esercizio 3 (punti 3)

Si descriva un diagramma delle classi UML per la seguente situazione. In una società che sviluppa software, quando si scopre un errore in un modulo software, viene generata una SegnalazioneDiMalfunzionamento, che contiene la descrizione del malfunzionamento e la data in cui esso si e` manifestato. Ogni progettista può segnalare malfunzionamenti e ogni malfunzionamento ha associato il progettista che l'ha segnalato. Un malfunzionamento ha un attributo che ne indica lo stato. Quando viene segnalato, il suo stato viene considerato "aperto". Per eliminare un malfunzionamento, gli viene assegnato un progettista responsabile della sua correzione. Una volta corretto, lo stato del malfunzionamento diventa "potenzialmente chiuso". Al malfunzionamento sono associati uno o più dati di test. Se questi vengono eseguiti con successo, lo stato del malfunzionamento diventa "chiuso".

1. Si descriva un Class Diagram che illustra le entita` in gioco e le relative associazioni.
2. Si descriva mediante un Sequence Diagram uno scenario in cui un particolare progettista esamina un malfunzionamento che si trova nello stato "potenzialmente chiuso", esegue i test associati al malfunzionamento e, nel caso questi vengano eseguiti con successo, dichiari il malfunzionamento "chiuso".

Soluzione:



